

Beyond Passing Tests: LLM-Driven Quality Control for Automated Program Repair

Seng Aloun Lorvanh
Dept. of Computer Engineering
Sangji University
Wonju-si, South Korea
0009-0008-1084-0800

Muhammad Muneeb
Dept. of Computer Engineering
Sangji University
Wonju-si, South Korea
0009-0006-0187-4406

Kwang-Man Ko
Dept. of Computer Engineering
Sangji University
Wonju-si, South Korea
0000-0002-7465-5400

Abstract—Most LLM-based Automatic Program Repair (APR) techniques judge patch correctness primarily by test-suite pass rates, leaving them vulnerable to overfitting patches that satisfy available tests without resolving the underlying defect. We present a post-generation quality-control framework that combines (1) LLM-based semantic assessment, (2) counterexample-driven overfitting detection supported by static analysis, and (3) preference-calibrated multi-criteria ranking over correctness, readability, minimality, and security. Each candidate is scored using a Patch Validity Score (PVS) that combines semantic correctness, counterexample pass rate, edit size, and vulnerability evidence. On a stratified sample of 150 Defects4J faults, the prototype achieves an overfitting-detection F1 of 0.81, Precision@1 of 0.75, and a counterexample success rate of 0.80. Average latency is 34 s, slightly exceeding the 30 s target. Rather than proposing a new patch generator, this work investigates integrated post-generation quality control for APR.

Index Terms—automatic program repair, large language models, overfitting patch detection, patch ranking, preference learning

I. INTRODUCTION AND RESEARCH QUESTIONS

Large language models (LLMs) have substantially improved patch-generation capability for Automatic Program Repair (APR) [1], yet the dominant plausibility criterion – passing a fixed test suite – allows *overfitting patches* that satisfy tests without fixing the true defect [2]–[4]. Post-hoc verification is often absent or limited to static rules, while patch ranking remains dominated by test-pass status, under-representing readability, minimality, and security [3], [2].

We therefore target the post-generation quality-control stage—semantic assessment, overfitting detection, and multi-criteria ranking—rather than patch generation itself. We investigate four research questions:

- **RQ1.** Does dual verification (LLM-based semantic assessment + counterexample generation) improve overfitting-detection F1 over test-based and static baselines?
- **RQ2.** Does preference-calibrated multi-criteria ranking improve top-1 patch selection relative to ChatRepair and entropy-based ranking?
- **RQ3.** How do counterexample generation and static analysis individually contribute to overfitting detection, and are their effects complementary?
- **RQ4.** Is the processing latency acceptable for CI/CD deployment, and where is the primary efficiency bottleneck?

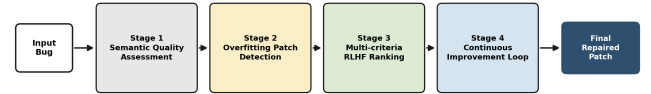


Fig. 1. Overall pipeline: four post-generation quality-control stages.

II. RELATED WORK AND CONTRIBUTION

Prior APR work has advanced individual components of this problem. TBar [5] relies on manually defined repair templates without; AlphaRepair [6] and ChatRepair [7] focus on patch generation; LLM4PatchCorrect [2] evaluates correctness but does not integrate it with re-ranking or overfitting removal; entropy-guided ranking [3] improves candidate selection; PRISM [4] targets overfitting detection.

In contrast, we integrate (i) semantic quality assessment, (ii) counterexample-based overfitting detection, and (iii) preference-calibrated multi-criteria ranking into a unified post-generation quality-control pipeline. The contribution is therefore not a new patch generator, but an integrated framework that jointly addresses overfitting detection and candidate ranking, which prior APR studies have largely treated separately.

III. PROPOSED FRAMEWORK

Fig. 1 shows the four-stage pipeline. **Stage 1** estimates a semantic quality score $S_{sem} \in [0, 1]$ from source code and available natural-language context, including bug reports, commit messages, and comments. **Stage 2** generates LLM-driven counterexamples targeting boundary conditions and cross-checks candidate behavior with CFG/PDG-based static analysis. Candidates are evaluated using:

$$PVS(p) = \alpha S_{sem}(p) + \beta S_{cex}(p) - \gamma S_{edit}(p) - \delta S_{vuln}(p) \quad (1)$$

where S_{cex} is the counterexample pass rate, S_{edit} penalizes large modifications, and S_{vuln} reflects statically detected vulnerability patterns.

Stage 3 learns $(\alpha, \beta, \gamma, \delta)$ using 240 pairwise developer preferences from three developers with a Bradley–Terry preference objective, ranking surviving candidates by correctness, readability, minimality, and security. **Stage 4** closes the loop through CI/CD-integrated certification and feedback collection for future recalibration.

TABLE I
PRELIMINARY RESULTS ON 150 DEFECTS4J FAULTS.

Metric	Baseline	Prior best	Entropy	Proposed
Overfitting F1	0.61	0.75 (ODS)	0.78	0.81
Precision@1	0.58	0.72 (CR)	0.74	0.75
Counter-ex. rate	–	0.79 (ES)	–	0.80
Avg. time (s)	8	20–90	25	34

TABLE II
ABLATION: OVERFITTING-DETECTION MODULES.

Configuration	Precision	Recall	F1
Counterexample only	0.68	0.86	0.76
Static analysis only	0.84	0.65	0.73
Combined (proposed)	0.80	0.82	0.81

IV. EXPERIMENTAL SETUP

We stratified-sampled 150 faults from Defects4J v2.0 [8] Chart, Lang, Math, Time, Closure, pooling 612 candidate patches from AlphaRepair, ChatRepair, and GPT-4o-mini. Overfitting labels combine semantic comparison with developer patches and previously reported patch-correctness labels.

Baselines include test-based pass/fail (ODS) [9], ChatRepair’s default selection strategy, and a reimplemented entropy-based ranker [3]. All methods use on identical hardware (1×NVIDIA A100 40GB, 32-core CPU) over the same candidate pool, ensuring a controlled and objective comparison.

V. RESULTS AND KEY FINDINGS

Table I reports the main comparison; Table II isolates each detection module’s contribution.

RQ1. Dual verification reaches F1 0.81, compared with 0.61 for the test-based baseline and 0.75 for ODS. These results show an observed improvement on the evaluated sample, although statistical significance is not established in the current study.

RQ2. Preference-calibrated ranking attains Precision@1 of 0.75, compared with ChatRepair at 0.72 and entropy-based ranking at 0.74. The method therefore achieves competitive top-1 selection, although the small margin over entropy ranking requires larger-scale statistical evaluation before establishing a conclusive advantage.

RQ3. Counterexample generation favors recall (0.86), whereas static analysis favors precision (0.84). Their combination yields the highest F1 (0.81), indicating complementary error profiles. Performance is highest for Math and Time (F1 0.83) and lower for Closure (F1 0.78). One possible explanation for the Closure result is information loss from context slicing, although the current experiment does not isolate this factor.

RQ4. Average latency is 34 s, narrowly missing the 30 s target while remaining within the 20–90 s range reported for the ODS comparison. The sequential static-analysis and LLM-inference stages are candidates for future profiling and optimization.

A. Threats to Validity

Internal validity is limited by partial reuse of patch-correctness labels from prior literature, which may propagate labeling inconsistencies. External validity is limited by evaluation on Java and a 150-bug Defects4J sample. Construct validity is affected by calibration of four PVS weights from only 240 pairwise preferences from three developers. Conclusion validity is limited by the single-seed evaluation and absence of significance testing. These limitations motivate larger-scale, multi-seed evaluation.

VI. CONCLUSION AND FUTURE WORK

We presented an integrated post-generation quality-control framework combining semantic assessment, counterexample-driven verification, static analysis, and preference-calibrated patch ranking. On 150 Defects4J faults, the combined method achieves overfitting-detection F1 of 0.81 and Precision@1 of 0.75, while requiring an average of 34 s per candidate. The ablation suggests that counterexample generation and static analysis provide complementary recall- and precision-oriented signals.

Future work will extend evaluation to the full Defects4J benchmark and multilingual datasets, perform multi-seed statistical evaluation, increase developer-preference data, and optimize the verification pipeline for CI/CD deployment.

REFERENCES

- [1] S. Lee *et al.*, “A Survey of LLM-based Automated Program Repair: From Single Model to Agent-based Pipelines,” *J. Softw. Eng. Res.*, 2024.
- [2] H. Zhang *et al.*, “LLM4PatchCorrect: Improving Program Repair with Large Language Model-based Patch Correctness Assessment,” *arXiv preprint*, 2023.
- [3] J. Wang *et al.*, “Entropy-guided Patch Ranking for Automated Program Repair,” in *Proc. ASE*, 2024.
- [4] M. Zhao *et al.*, “PRISM: Semantic-aware Overfitting Patch Detection for Automated Program Repair,” in *Proc. ICSE*, 2025.
- [5] K. Liu *et al.*, “TBar: Revisiting Template-based Automated Program Repair,” in *Proc. ISSSTA*, 2019.
- [6] C. S. Xia and L. Zhang, “Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-shot Learning,” in *Proc. FSE*, 2022.
- [7] C. S. Xia and L. Zhang, “Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT,” in *Proc. ISSSTA*, 2023.
- [8] R. Just *et al.*, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” in *Proc. ISSSTA*, 2014.
- [9] G. Fraser and A. Arcuri, “EvoSuite: Automatic Test Suite Generation for Object-oriented Software,” in *Proc. FSE*, 2011.
- [10] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “GenProg: A Generic Method for Automatic Software Repair,” *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, 2012.
- [11] H. Park *et al.*, “Security Vulnerability Repair via Large Language Models,” in *Proc. ISSSTA*, 2024.